

# CaseWise AI

**Complete Project Documentation**

# Project Overview

## CaseWise AI

CaseWise AI is a local development implementation of a legal case workspace with case-scoped document retrieval. AI-generated information should be reviewed and verified by a qualified legal professional.

### Run with one command

Install Docker Desktop, start its engine, then run from this directory:

```
docker compose up --build
```

Open **http://localhost:3000**. The default command starts a local PostgreSQL case database with its schema and RLS, then the FastAPI and Next.js services. The ignored `.env` configures hosted Supabase Auth and Storage, hosted Qdrant, and Gemini. The BGE embedding model downloads when the first document is processed. These downloads need internet and can take time.

The hosted PostgreSQL URI remains in `.env` for later use. Both the Session pooler (port 5432) and Transaction pooler (port 6543) failed from the user's Docker network before authentication, so the default app does **not** use hosted Supabase PostgreSQL. Case records stay in the local Docker volume. [The hosted migration](supabase/migrations/20260929\_casewise\_hosted.sql) is retained but not applied. The Supabase secret key is stored only in `.env` and used server-side for private document Storage.

When PostgreSQL TCP egress is restored, the hosted migration can be run explicitly with `docker compose --profile hosted-db run --rm migrate`. Configure the API to use the hosted database only after that migration succeeds. Passwords in a URI must stay URL-encoded (for example, `@` becomes `%40`). The Python PostgreSQL client disables prepared statements for transaction pooler compatibility.

Create a lawyer account in the app. A lawyer can register a case, reuse or create a client, upload PDF/DOCX/TXT documents, manage case records, and ask a document-grounded question once a document shows READY. To give a client access, have them register with the email saved on the client record, then use **Grant client access** in the case overview.

## Services

Service | Local address | Purpose

--- | --- | ---

Web | <http://localhost:3000> | Next.js UI and API proxy

AI/API | <http://localhost:8000/docs> | FastAPI case services and OpenAPI docs

Auth | Supabase project URL | Hosted Supabase Auth

Qdrant | Cloud endpoint in ``.env`` | Case-filtered vector index

Gemini | Google API | Grounded answers and case review

PostgreSQL | Docker internal (default) | Case data and RLS

Docker volumes retain case records and the embedding cache. With the supplied Supabase secret key, original documents are kept in a private case-document's Supabase Storage bucket and served only through the authorized CaseWise API. `docker compose down` keeps local volumes; cloud data remains in the respective projects.

## Implemented code (runtime verification pending)

- Supabase email/password authentication with lawyer and client profiles.
- PostgreSQL case, client, membership, document, claims, evidence, timeline, hearing, task, payment tracking, note, analysis, and audit tables with RLS policies.
- Lawyer case registration, existing/new client choice, opposing party, private documents, and basic case records.
- Client access through an explicit case grant; client visibility restrictions for case records.
- PDF/DOCX/TXT text extraction, local BGE embeddings, Qdrant indexing with exact case ID payloads and query filters.
- Gemini-backed Ask CaseWise with verified source IDs and clickable document citations. When sources or models are unavailable, the UI shows a real error or insufficient-information response.
- An on-demand AI case review that retains only suggestions containing an exact quote from an authorized source chunk; its sections cover timeline, claims, evidence, possible gaps, potential contradictions, counterarguments, and hearing preparation.
- A read-only MCP adapter in `services/mcp`` that reuses the authenticated API. It is intended to launch per user with `CASEWISE_USER_TOKEN`` and is not enabled in the default Compose profile.

## Current limits

This is a development build, not production legal software. PDF scanning/OCR, durable job retries, automatic analysis after registration, structured transfer of AI review suggestions into editable case records, reminders, notification delivery/UI, and export still need implementation. The on-demand AI review is experimental and can return empty sections rather than unsupported claims. The Register action processes documents, but it does not claim analysis has finished. Replace development database credentials and configure secure session handling before deployment. Browser sessions currently use local storage with token refresh. Hosted Storage behavior still needs runtime verification.

Docker startup could not be exercised in this workspace because Docker daemon access was denied. `docker compose config` and Python syntax checks passed. See [project context](CASEWISE\_PROJECT\_CONTEXT.md) and [roadmap](docs/development-roadmap.md) for the remaining work.

# Project Context

## CaseWise project context

### Current implementation

The repo has a Docker Compose development stack: local PostgreSQL, Next.js web app, and FastAPI case/AI service. Its ignored `.env` points at hosted Supabase Auth/Storage and Qdrant, and selects Gemini for generation. Hosted Supabase PostgreSQL direct, Session pooler, and Transaction pooler connections were unreachable from the user's Docker network; the default command now uses local PostgreSQL for case records. The guarded hosted migration is an optional Compose profile. Local GoTrue, Qdrant, and Ollama remain optional profiles. The user has built the app images, but application startup and workflows still need verification. The root [README](README.md) describes setup and current limits.

Implemented code covers lawyer/client signup and sign-in; case and client records; case membership and client grants; register case UI; file upload and PDF/DOCX/TXT extraction; local BGE embeddings; Qdrant case-scoped retrieval; Ollama-backed cited answers and on-demand quoted-source case review; basic manual claims, evidence, timeline, hearings, tasks, payment tracking, notes; and a separate read-only MCP adapter with document search. Runtime behavior still needs Docker verification.

### Architecture and decisions

- ``apps/web``: Next.js/React/JavaScript UI. Its API route proxies authenticated case requests to FastAPI. Browser authentication calls hosted Supabase Auth directly using the publishable key; Supabase private keys stay server-side.
- ``services/ai``: FastAPI owns case services, document extraction, indexing, and grounded AI. It validates GoTrue tokens through ``/user``, uses the user's UUID in PostgreSQL transaction claims, and checks case membership before operations.
- ``supabase/init/01-schema.sql``: local development schema, applied to a fresh Docker PostgreSQL volume. ``supabase/migrations/20260929_casewise_hosted.sql`` is the hosted migration; the optional migration service checks for existing tables before applying it transactionally. It has not been applied.
- ``services/mcp``: separate per-user stdio MCP server. Tools call the same FastAPI case API using a user access token from its environment; it has no write tools.
- Qdrant stores chunk embeddings with ``caseId``, document metadata, and chunk ID. Queries always include exact case ID filtering, then recheck chunks/documents in PostgreSQL.
- Gemini is the selected LLM provider through an abstraction; Ollama remains available as an optional local provider. BGE embeddings remain local. A Gemini API key is required for generated answers and case review.

- Private document bytes use a private Supabase Storage bucket when the hosted URL and secret key are configured; otherwise local development falls back to a Docker volume. Storage requests occur server-side after case authorization. Hosted Storage behavior has not been runtime verified.

## Important pending work

- Verify local PostgreSQL initialization, API and web startup, and real user workflows. Investigate blocked outbound TCP to Supabase PostgreSQL before any hosted database switch.
- Add automated integration tests for auth, RLS, cross-case Qdrant access, and client-private data boundaries.
- Implement durable jobs, retries, deletion/reindex consistency, and Supabase Storage policies.
- Turn on-demand quoted-source AI review into reliable, editable timeline/claim/evidence records and richer gap, contradiction, counterargument, and hearing preparation workflows. The current review is experimental and can return empty sections.
- Add reminders, notification delivery, richer case editing, AI result editing, and production session/security hardening.

The architecture documents in docs/ are the target design. They include planned capabilities that are not yet implemented; use this status file and README to distinguish shipped code from design.

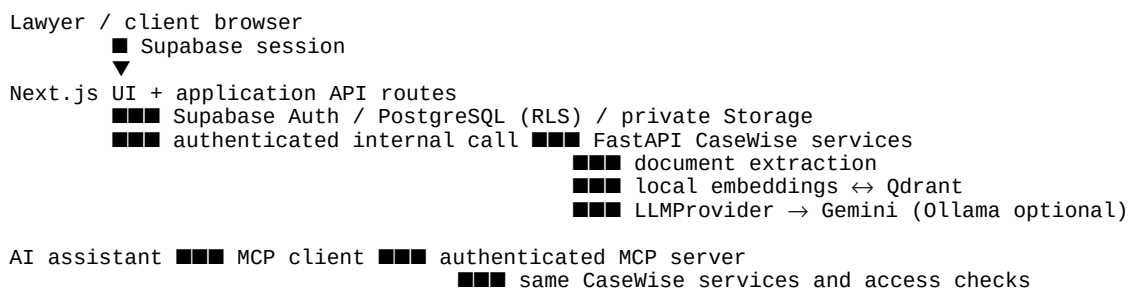
# Architecture

## Architecture

### Repository inspection

The repository was empty at the start of phase 1. A local development stack has since been added. The design below remains the target architecture; current implementation and remaining gaps are tracked in [project context](../CASEWISE\_PROJECT\_CONTEXT.md).

### Planned system



PostgreSQL is authoritative for identity, case membership, structured records, processing state, and visibility. Qdrant is a derived search index and must never grant access by itself. FastAPI owns document and AI workflows. Next.js owns user-facing APIs and UI. MCP adapts authenticated service capabilities into tools without duplicating business logic.

### Boundaries

- **\*\*Browser to Next.js:\*\*** Supabase session; validate inputs and user access on the server for every route.
- **\*\*Next.js to FastAPI:\*\*** authenticated internal request carrying verified user identity and case ID; FastAPI rechecks case access against PostgreSQL. Never trust a browser-supplied role.
- **\*\*FastAPI to Qdrant:\*\*** require a case ID obtained from an authorized request; exact-match case filter on every read. Revalidate returned payloads and source records.
- **\*\*MCP to services:\*\*** each invocation has a verified end-user principal and scoped case access. Write tools, when introduced, require explicit user confirmation and audit logging.

### Folder structure to grow into

|                      |                                          |
|----------------------|------------------------------------------|
| apps/web/            |                                          |
| app/(auth)/          | sign-in and sign-up                      |
| app/(lawyer)/        | lawyer dashboard and register-case flow  |
| app/(client)/        | client dashboard                         |
| app/api/             | normal application APIs                  |
| components/          | shared and feature-specific UI           |
| lib/auth/            | Supabase session helpers                 |
| lib/services/        | server-side case services and AI client  |
| lib/validation/      | request schemas                          |
| services/ai/         |                                          |
| app/api/             | FastAPI routers                          |
| app/auth/            | principal and case authorization         |
| app/services/        | ingestion, retrieval, analysis           |
| app/providers/       | LLM and embedding interfaces/adapters    |
| app/models/          | request and response schemas             |
| tests/               | security and pipeline tests              |
| services/mcp/        |                                          |
| app/                 | MCP server and thin tool adapters        |
| tests/               | authorization tests                      |
| supabase/migrations/ | schema and RLS migrations                |
| infra/qdrant/        | local configuration and collection setup |
| docs/                | architecture and API contracts           |

These nested folders are a design, not a claim that their code exists. Add them as their corresponding phase begins.

## Initial vertical slice

Authenticate a lawyer, register a case with an authorized client association, upload one private document, extract and index it, then answer one case-specific question with a verified citation. Expose structured timeline, claims, and evidence only after that path is reliable.

# API Documentation

## API design

This document describes proposed routes; none are implemented yet. JSON contracts should be versioned as they stabilize. All normal application routes authenticate the Supabase session, validate inputs, and perform server-side case authorization.

### Next.js application routes

Route | Planned operations | Access

--- | --- | ---

`/api/cases` | List, create draft/register case | Lawyer for create; role-scoped list

`/api/cases/[id]` | Read/update case | Assigned member read; lawyer update

`/api/cases/[id]/documents` | List/upload documents | Role-visible list; lawyer upload

`/api/cases/[id]/documents/[documentId]` | Metadata and signed source URL | Authorized for that document

`/api/cases/[id]/claims` | List/review claims | Role-visible list; lawyer edit

`/api/cases/[id]/evidence` | List/review evidence | Role-visible list; lawyer edit

`/api/cases/[id]/timeline` | Sourced events | Role-visible

`/api/cases/[id]/hearings` | List/manage hearings | Role-visible list; lawyer edit

`/api/cases/[id]/tasks` | List/manage tasks | Assigned tasks; lawyer management

`/api/cases/[id]/payments` | Tracking | Explicitly shared client view; lawyer edit

`/api/notifications` | List/read own notifications | Recipient only

`/api/cases/[id]/ask` | Grounded question and citations | Authorized document scope

Use a transaction for case registration and client association. Uploaded files enter a separate retryable pipeline; the response returns real job/document statuses. Validate IDs, pagination, file type/size, and bounded question length. Return 401 for missing session, 403/404 without revealing unauthorized case details, 422 for invalid input, and actionable processing errors.

### FastAPI internal routes

Route | Purpose

--- | ---

`/ai/process-document` | Start or retry authorized document ingestion

`/ai/query-case` | Case-scoped retrieval and grounded answer

`/ai/analyze-case` | Orchestrate persisted analysis suggestions

`/ai/timeline` | Extract sourced timeline candidates

`/ai/claims` | Extract sourced claim candidates

`/ai/evidence-gaps` | Find possible gaps from claims and mapped evidence

`/ai/contradictions` | Find potential source conflicts

`/ai/counterarguments` | Produce sourced potential arguments

`/ai/hearing-prep` | Draft an editable preparation brief

These are internal authenticated endpoints, not public model access. Long jobs should return a job ID and status rather than block a request. Route handlers call service functions that also back MCP tools; embed/index functions can remain internal service calls unless there is a concrete need for HTTP endpoints.

## Response shapes

An answer should contain `answer`, `evidence[]`, `citations[]`, `potentialIssues[]`, `suggestedReview[]`, and `status`. Each citation includes `documentId`, `documentName`, `pageNumber` or `null`, `chunkId`, and an authorized source route. Processing status includes completed steps and an error code/message when failed. Never return fabricated citations or optimistic completion states.

# Database Schema

## Database design

This is the proposed PostgreSQL schema, not an applied migration. Use UUID primary keys, `created_at` / `updated_at` timestamps, foreign keys, bounded status values, and indexes for common case-scoped queries. `auth.users` owns authentication; `profiles.id` references it.

### Core relationships

Table | Main columns | Notes

--- | --- | ---

``profiles`` | ``id``, ``role``, ``display_name``, ``email`` | ``id`` → `auth.users.id`; role ``LAWYER`` or ``CLIENT``. Email is display data, not the authorization key.

``clients`` | ``id``, ``kind``, ``name``, ``phone``, ``email``, ``address``, ``portal_user_id`` | ``portal_user_id`` → `profiles.id`` nullable until invitation is accepted. Lawyer-created client records can exist without login.

``cases`` | ``id``, ``title``, ``case_number``, ``case_type``, ``court``, ``status``, ``priority``, ``filing_date``, ``client_id``, ``main_issue``, ``description``, ``created_by`` | ``client_id`` → `clients.id``; draft or active status.

``case_members`` | ``case_id``, ``user_id``, ``member_role``, ``created_at`` | Composite PK. The explicit lawyer/client access grant; link a client login only after verification.

``opposing_parties`` | ``id``, ``case_id``, ``kind``, ``name``, ``contact``, ``address``, ``counsel`` | A case may have multiple opposing parties.

``documents`` | ``id``, ``case_id``, ``file_name``, ``mime_type``, ``byte_size``, ``storage_path``, ``processing_status``, ``visibility``, ``uploaded_by`` | Private Storage path is server controlled; status records real pipeline progress/failure.

``document_chunks`` | ``id``, ``case_id``, ``document_id``, ``page_number``, ``section``, ``text``, ``content_hash``, ``embedding_state`` | PostgreSQL provenance; Qdrant point ID matches this row ID.

``claims`` | ``id``, ``case_id``, ``statement``, ``source_chunk_id``, ``status``, ``visibility``, ``reviewed_by`` | AI suggestions are editable and distinguishable from reviewed facts.

``evidence`` | ``id``, ``case_id``, ``name``, ``type``, ``description``, ``source_document_id``, ``source_page``, ``status``, ``visibility``, ``notes`` | Source points to a case document.

``claim_evidence`` | ``claim_id``, ``evidence_id``, ``relationship``, ``created_at`` | Composite PK; both records must belong to the same case.

``timeline_events`` | ``id``, ``case_id``, ``event_date``, ``description``, ``source_chunk_id``, ``status``, ``visibility`` | Store uncertainty/precision if the source gives only a partial date.

``contradictions`` | ``id``, ``case_id``, ``source_a_chunk_id``, ``source_b_chunk_id``, ``description``, ``review_status``, ``visibility`` | Potential inconsistency; neither source is declared false.

``hearings` | `id`, `case_id`, `starts_at`, `court`, `purpose`, `status`, `notes`, `visibility`` | Reminders in a related schedule table or constrained JSON once worker design is set.

``tasks` | `id`, `case_id`, `title`, `assignee_id`, `due_at`, `status`, `visibility`` | Client assignees see only shared tasks.

``payments` | `id`, `case_id`, `total_fee`, `amount_paid`, `due_date`, `status`, `visibility`` | Tracking only; no payment credentials or processing.

``notifications` | `id`, `recipient_id`, `case_id`, `kind`, `message`, `read_at`` | Recipient scoped.

``ai_analyses` | `id`, `case_id`, `kind`, `result`, `source_chunk_ids`, `status`, `visibility`, `model`` | Generated results and provenance; private by default.

``case_notes` | `id`, `case_id`, `author_id`, `body`, `visibility`` | Lawyer private by default.

``activity_logs` | `id`, `case_id`, `actor_id`, `action`, `entity_type`, `entity_id`, `created_at`` | Append-only audit; no sensitive document text in logs.

## Constraints and indexes

- Enforce ``case_members(case_id, user_id)`` uniqueness and indexes on ``user_id, case_id``.
- Index every case child table on ``case_id``; index dates/statuses used by dashboards (``hearings.starts_at`, `tasks.due_at`, `documents.processing_status``).
- Enforce same-case source references for chunk, claim, evidence, and analysis links through composite foreign keys or database triggers; an ordinary foreign key to an ID alone does not prove same-case ownership.
- Use check constraints or carefully migrated enums for roles, visibility, processing status, and task/hearing status.
- Store money in a fixed decimal type with currency; enforce nonnegative amounts. Compute remaining balance from recorded amounts.
- Keep immutable original filenames for display, but use generated IDs in private storage paths.

## RLS strategy

Enable RLS on every user-visible table and private storage bucket. Define narrowly scoped SQL helper functions such as `is_case_lawyer(case_id)` and `is_case_client(case_id)` based on `auth.uid()` and `case_members`; review their `search_path` and privileges. A lawyer sees assigned case records. A client sees their assigned case and only rows explicitly marked `CLIENT_SHARED` (plus their own tasks/notifications where appropriate). `case_members` edits must go through a controlled server operation that checks grant authority; users cannot self-enroll. Inserts and updates require matching `WITH CHECK` policies so records cannot be moved between cases. Service-role use is limited to trusted workers and must perform explicit authorization before accessing user data.

See [security.md](security.md) for policy details and Storage access.

# AI Features

## Product and AI feature plan

AI-generated information should be reviewed and verified by a qualified legal professional. The platform must not predict outcomes, assign win probabilities, give guaranteed legal advice, or autonomously make legal decisions.

### Register New Case

Build a desktop-first, responsive multi-section page for lawyers:

- **Case information:** title, type, number, court, status, filing date, priority.
- **Client details:** select an existing client or create an individual/organization with name, phone, email, address. Show which mode is active and avoid duplicate client creation.
- **Opposing party:** name, kind, contact, address, opposing counsel.
- **Case details:** main issue and description.
- **Important dates:** filing date, next hearing, document deadline, payment due date, extra dated events.
- **Initial documents:** drag-and-drop PDF/DOCX/TXT, file validation, per-file upload/processing/processed/failed states.

Actions are **Save as Draft** and **Register Case & Analyze**. The latter creates the case, uploads files, starts real processing jobs, and displays each actual step: case created, documents uploaded, text extracted, chunks embedded/indexed, timeline and claims suggested, evidence mapped, gaps and contradictions checked, analysis completed. Failed steps stay visible with retry where safe. The dashboard opens when the required pipeline reaches a reliable state; partial results are labeled.

### Case views

The lawyer dashboard emphasizes active cases, upcoming hearings, pending tasks, payment dues, possible evidence gaps, potential contradictions, and recent activity. The client dashboard shows only their case status, next hearing, requested documents, assigned tasks, shared payment status, and notifications. Case details use tabs for Overview, Timeline, Documents, Evidence, Claims, AI Analysis, Hearings, Tasks, Payments, and Notes; client navigation omits inaccessible content.

### Ask CaseWise

Render a concise answer, relevant evidence, clickable citations, potential issues, and suggested review areas. Answers come only from authorized case documents. If retrieval is weak or unavailable, show the explicit insufficient-information response. Citations point to the authorized document and page when page information exists.

## Structured intelligence

- **Timeline:** event date, description, document/page source, uncertainty; show conflicting dates as potential inconsistencies for lawyer verification.
- **Claims and evidence:** editable claim/evidence records, source linkage, and mapping status. Separate extracted suggestions from lawyer-reviewed records.
- **Possible evidence gaps:** identify claims whose uploaded support appears incomplete; do not imply a legal outcome.
- **Potential contradictions:** show both cited sources and request verification without deciding which is true.
- **Potential counterarguments:** explicitly separate fact, inference, and potential argument, with sources.
- **Hearing preparation:** editable brief with date, court, issues, facts, evidence, gaps, contradictions, potential counterarguments, and documents to review.
- **Case preparation readiness:** coverage indicators for documents, claims/evidence, timeline consistency, counterargument review, and hearing preparation. Explain that these measure preparation coverage, not outcome likelihood.

## Other workflows

Hearings have date/time, court, purpose, status, notes, and configurable reminders (7, 3, 1 day, or custom). Tasks can be assigned to lawyers or clients and tracked as pending, in progress, or complete. Payments track total, paid, remaining, due date, and status only. Notifications cover hearings, payment due dates, document requests, tasks, processing completion, gaps, and inconsistencies.

## Fictional demo fixture

Use **ABC Traders vs XYZ Enterprises**, C.S. 245/2026, Commercial Court, Coimbatore, only as labeled fictional fixture data. The prompt lists Agreement, Invoice, Delivery Receipt, Payment Request, Legal Notice, and Communication documents, but provides no actual contents. Do not invent their text, events, citations, or extracted evidence. Add real synthetic fixture files deliberately when test cases are written.

# RAG Implementation

## Document processing and grounded RAG

### Ingestion

- Next.js checks lawyer membership and validates PDF, DOCX, or TXT upload. Store the original in a private Supabase bucket and a `documents` row with `UPLOADED` state.
- A durable worker job moves through `VALIDATING`, `EXTRACTING`, `CHUNKING`, `EMBEDDING`, `INDEXING`, and `READY`, or records `FAILED` with a safe error. Do not show completed steps before they finish.
- Use PyMuPDF for page-aware PDF text, `python-docx` for DOCX, and a bounded decoder for TXT. DOCX and TXT may have no reliable page number; represent that as `null`, never invent page references. Empty/scanned PDFs report `OCR\_REQUIRED`; OCR is a later feature.
- Chunk per page or logical section with controlled overlap. Persist chunk text and source metadata in PostgreSQL. Embed locally using a configured Sentence Transformers/BGE model.
- Upsert Qdrant points using the chunk UUID. Required payload: `caseId`, `documentId`, `documentName`, `pageNumber` (nullable), `section` (nullable), `chunkId`. Also store content/version metadata for reindexing.

The database and vector index can diverge. Keep a retryable indexing state, idempotent point IDs, and a reconciliation job. Deleting a document revokes database access immediately and queues vector deletion; retrieval also checks the database so stale points cannot leak.

### Retrieval and answer

```
verified principal + authorized case ID + question
→ local question embedding
→ Qdrant exact caseId filter
→ score threshold and diversity selection
→ recheck each chunk and document in PostgreSQL
→ context with stable source IDs
→ LLMProvider (Gemini selected; Ollama optional)
→ validate cited source IDs and answer shape
→ cited response or insufficient-information response
```

If there is no sufficient source material, respond: **"I could not find sufficient information in the uploaded case documents."** Do not fill missing facts from model memory. Distinguish source fact, inference, and potential argument. Source citations should include document name, page when available, and a source route that checks access before opening the document. Prompt injection in documents must be treated as untrusted text; document instructions cannot change the assistant's tool or access rules.

## Provider interfaces

LLMProvider supports answer generation and structured output. GeminiProvider reads GEMINI\_API\_KEY, GEMINI\_ANSWER\_MODEL, and GEMINI\_ANALYSIS\_MODEL from central configuration; OllamaProvider remains an optional local adapter. EmbeddingProvider is separate; the local BGE adapter reads EMBEDDING\_MODEL. Changing embedding models requires a new compatible Qdrant collection or full reindex; vector dimensions are fixed per collection.

LangChain may help with parsing/retrieval but is optional. LangGraph is only justified by a multi-step workflow with meaningful state and recovery. A simple case question should remain a direct service flow.

## Structured analysis

Timeline events, claims, evidence mappings, gaps, contradictions, counterarguments, and hearing briefs should be persisted as suggestions with source chunk IDs, confidence/uncertainty, visibility, and review status. Lawyers can edit and approve them. A generated item without verifiable source IDs must be rejected or explicitly labeled as an unsupported suggestion. No outcome probability is computed.

# Security

## Security and authorization

### Identity and role

Supabase Auth issues the user session. `profiles.role` is server-managed and never accepted from a client request as an authority claim. Case access comes from `case_members`, not a case ID in a URL or a vector payload. Invitation and membership changes are privileged operations with audit entries.

### Access matrix

Resource | Assigned lawyer | Assigned client | Other user

--- | --- | --- | ---

Case metadata | Read/update according to role | Read client-safe fields | No access

Documents | Read/manage | Read only ``CLIENT_SHARED`` | No access

Claims, evidence, timeline | Read/review | Read only ``CLIENT_SHARED`` | No access

Internal AI analyses, counterarguments, hearing preparation | Read/manage | No access unless explicitly shared | No access

Lawyer notes | Read/manage | No access unless explicitly shared | No access

Client tasks and notifications | Read if case member | Read own or shared records | No access

Payments | Read/manage | Read only explicitly shared status | No access

An assigned lawyer is a `case_members` row with `member_role = LAWYER`; an assigned client is a row with `member_role = CLIENT`. The client row must match the verified `clients.portal_user_id` for the case's client, unless a later multi-client design explicitly permits more. Keep client-safe case data in a view or dedicated response serializer so private columns cannot slip into API responses.

### RLS policy rules

- Enable and test RLS for **every** case-related table, ``profiles``, ``clients``, and Storage objects. Default deny; define explicit ``SELECT``, ``INSERT``, ``UPDATE``, and ``DELETE`` rules where needed.
- For lawyer access, require ``auth.uid()`` to have an active lawyer membership on the same ``case_id``. For client access, require client membership and ``visibility = CLIENT_SHARED`` on records with private content.

- ``USING`` and ``WITH CHECK`` must both constrain case ID, membership, and role. Guard against a user changing ``visibility``, ``case_id``, ``created_by``, or role to escalate access.
- Policies for join tables must verify both linked records belong to the same authorized case.
- Notifications are addressed to ``auth.uid()``; activity logs are readable only by assigned lawyers and writable through trusted server code.
- Do not expose the Supabase service-role key to browser code. Trusted workers using it must manually check the originating principal and case access.

## Storage and retrieval

Use a private Supabase Storage bucket. A server-generated path such as `cases/{case_id}/{document_id}/{safe_name}` is metadata, not proof of access. Storage read policies should join document records and case membership; clients also need shared visibility. Prefer short-lived signed URLs generated after authorization. Validate MIME type by content, extension, and file size before accepting PDF/DOCX/TXT. Reject executable or malformed content and record processing errors without leaking internals.

Every Qdrant query requires an exact-match `caseId` filter for a case the principal is authorized to read. The retrieval service must check returned `caseId`, document ID, document visibility, and page provenance against PostgreSQL before using chunks or producing citations. Never offer an unrestricted cross-case search endpoint. Qdrant is not an access control system.

## Internal services and MCP

Next.js, FastAPI, and MCP need a verified end-user principal and a protected service-to-service channel. FastAPI and MCP recheck access rather than trusting a forwarded role or case ID. Do not put long-lived service credentials in model prompts. MCP tool calls receive only the minimum data needed; write tools require explicit user confirmation and audit events. Log access decisions and failures without full document contents.

## Required security tests before release

- Unassigned user cannot enumerate cases, documents, storage objects, or Qdrant matches.
- Client cannot see private notes, counterarguments, private analyses, or unshared documents, including via citations or MCP.
- Case A member cannot retrieve Case B chunks even by supplying Case B IDs or malicious filters.
- Users cannot add themselves to ``case_members`` or change ``visibility``/roles through direct Supabase calls.
- Invalid uploads, forged service requests, and expired sessions fail safely.

# MCP Integration

## MCP architecture

The MCP server is a separate interface for the CaseWise AI assistant. It exposes structured capabilities backed by the same CaseWise services used by application APIs. It must not duplicate SQL, retrieval, or analysis rules inside tool handlers.

```
CaseWise assistant → MCP client → authenticated MCP server
                                → CaseWise service functions
                                → Supabase / Qdrant / FastAPI
```

### Initial read-only tools

| Tool                    | Purpose                                      | Access                                            |
|-------------------------|----------------------------------------------|---------------------------------------------------|
| ---                     | ---                                          | ---                                               |
| `get_case`              | Client-safe or lawyer-complete case summary  | Case member; role-aware projection                |
| `list_case_documents`   | Authorized document metadata                 | Case member; visibility filtered                  |
| `search_case_documents` | Case-scoped semantic excerpts and source IDs | Case member; exact case filter and source recheck |
| `get_case_timeline`     | Sourced timeline events                      | Case member; visibility filtered                  |
| `list_case_evidence`    | Reviewed and suggested evidence              | Case member; visibility filtered                  |
| `get_upcoming_hearings` | Authorized hearing list                      | Case member; visibility filtered                  |

Other prompt-listed tools—claim evidence, evidence gaps, conflicts, contradictions, counterarguments, hearing briefs—should be added when the underlying service and records exist. The tool list is deliberately focused; avoid demo-only tools.

### Tool contract

Every tool receives the authenticated principal from transport context, validates role and case membership in the service layer, and returns bounded structured data with source IDs. A model-supplied caseId is only a requested scope, never an authorization claim. Search must pass an exact case filter to Qdrant and validate each result against PostgreSQL. Tool errors should reveal no private case existence to unauthorized users.

### Actions and confirmation

Keep read tools separate from future write/action tools (create\_case, update\_case, reminders, tasks, payment tracking). The assistant first presents the proposed action and asks the user to confirm its concrete details. Only after confirmation may the server execute it under the user's permissions and record an audit event. MCP itself should not autonomously file legal documents or decide strategy.

# Development Roadmap

## Development roadmap

### Current state

Phases 1–9 have initial development implementations for the web app, Supabase Auth, PostgreSQL/RLS, case registration, document processing, Qdrant, provider-based Gemini/Ollama generation, and cited questions. The read-only MCP adapter also has initial code. Hosted PostgreSQL TCP access failed from the user's Docker network on both pooler ports, so the default stack uses local PostgreSQL and retains an optional hosted migration. Full app startup and live integration behavior have **not** been verified. See [project context](../CASEWISE\_PROJECT\_CONTEXT.md) for an exact implemented/pending list.

Phase | Deliverable | Exit check

--- | --- | ---

2 | Next.js shell, Supabase client/server setup, sign-in, lawyer/client roles | Server recognizes a real session and controlled role

3 | PostgreSQL migrations, RLS, private Storage policies | Cross-user and cross-case access tests pass

4–5 | Clients, cases, Register New Case form | Lawyer saves a draft and registers an authorized case

6–7 | Private upload, extraction, status tracking | PDF/DOCX/TXT produce real text or clear errors

8–9 | Local embeddings, Qdrant, Ollama, grounded query | Authorized case-only answer with validated citation

10–11 | Read-only MCP tools and clickable citations | Tools enforce same access as app; source opens securely

12–16 | Timeline, claims, evidence, gaps, contradictions, counterarguments | Suggestions have sources and can be reviewed/edited

17–19 | Hearing briefs, tasks, hearings, notifications, payments, dashboards | Role-specific workflow works end to end

20–21 | Security hardening, integration tests, deployment, docs | Tenant isolation, recovery, and operating instructions verified

### MVP boundary

The requested MVP covers auth and roles; Supabase schema/RLS; case/client management; registration; private document upload and PDF/DOCX/TXT extraction; local embeddings/Qdrant/Ollama; case-scoped RAG with citations and Ask CaseWise; sourced timeline, claims, evidence, gaps, basic contradictions; a read-only core MCP server; and lawyer/client dashboards. Build this as several verified milestones, not one merge.

## Later features

Advanced counterargument analysis, hearing brief export, OCR, rich reminders, organization-level permissions, broader MCP actions, alternative LLM providers, and deeper readiness indicators can follow the stable MVP. Payment processing and autonomous legal actions remain out of scope unless explicitly requested.

## Packages to evaluate when implementing

Area | Likely packages | Why / decision point

--- | --- | ---

Web | `next`, `react`, `react-dom`, `@supabase/supabase-js`, `@supabase/ssr` | App and authenticated Supabase clients; choose compatible current versions

Web validation/UI | `zod`; a form library if needed | Shared input validation; use native form handling where sufficient

Python API | `fastapi`, `uvicorn`, `pydantic`, `supabase` | Internal service and typed contracts

Documents | `pymupdf`, `python-docx` | PDF/DOCX extraction; Python standard library for TXT

Search | `sentence-transformers`, `qdrant-client` | Local embeddings and case-filtered vector search

Local LLM | `httpx` or Ollama Python client | Configurable Ollama HTTP integration

MCP | Official Python MCP SDK | Separate tool server; confirm auth transport support

Tests | `pytest`, `httpx`; web testing package as needed | Authorization and pipeline integration checks

Do not install all packages at once. Inspect the code and select dependencies for each phase. LangChain is optional; LangGraph needs a concrete multi-step use case.

## Technical risks

- **Cross-case or client-private data leakage:** RLS, server-side membership checks, exact Qdrant case filtering, and source revalidation are mandatory release gates.
- **Unsupported AI claims:** weak retrieval, stale index points, and invented citations must fail closed or return insufficient information.
- **Document quality:** scanned PDFs need OCR; DOCX/TXT may have no page number. Show that limitation honestly.
- **Job reliability:** upload, extraction, indexing, and analysis need idempotency, retries, status, and cleanup after partial failures.
- **Local resource use:** embedding models, Ollama, and Qdrant may exceed small developer machines; benchmark a realistic document set.

- **Schema evolution:** role visibility and same-case foreign-key invariants must be designed before adding many dependent tables.
- **Legal domain review:** terminology and generated summaries should be reviewed by qualified professionals before production use.